

Parameter-Space Trajectory Metrics

Ben Meyers

Overview

Gradient descent is a physical process. Parameters move through a high-dimensional space under forces determined by the loss landscape, the architecture, and the update rule. Standard training diagnostics—loss curves, accuracy, gradient norms—are scalar projections of this movement that discard almost everything.

The framework below instruments the *trajectory* itself: how far did each parameter travel, in what direction, driven by which gradient sources, and how much of that travel was purposeful versus wasted? Three leverage metrics then weight those trajectories by different notions of parameter importance.

Two foundational tensions. Gross and net are not redundant, and neither are efficiency and productivity. The gap between gross path and net displacement is where geometric waste lives—motion that cancelled itself. The gap between directedness and loss improvement is where productive waste lives—motion that was internally consistent but extrinsically useless. The framework is organized around these two tensions.

The metrics decompose into two orthogonal qualities of movement:

- **Intrinsic quality** (Sections 1–2): was the motion internally consistent? Did parameters move in coherent directions, or did they oscillate and cancel? These metrics are geometric—they describe the shape of the trajectory in parameter space without reference to the loss landscape.
- **Extrinsic quality** (Section 3): was the motion productive? Did it actually descend the loss landscape? These metrics require loss evaluations. They are stacked on top of the intrinsic metrics as a multiplicative qualification layer: any intrinsic metric M admits a loss-qualified twin $M^* = M \cdot \Delta^*$, where Δ^* is the fractional loss improvement. The geometric metric tells you about the shape of the trajectory; the qualified metric tells you whether that shape was worth it.

This qualification principle applies uniformly. The intrinsic metrics stand on their own—a training run with high η_1 but low Δ^* has real information: the model moved purposefully, it just did not learn much. The qualified metrics add the extrinsic dimension. Both layers are always worth tracking.

Contents

1	Base Accumulation: Gross Path and Net Displacement	2
1.1	Per-Source Attribution	4
1.2	Per-Source Per-Parameter Efficiency	4
2	Steepness: The Extrinsic Dimension of Movement	5
2.1	The Core Observation	5
2.2	Two Steepnesses	5
2.3	The Steepness-Directedness Decomposition	6
3	The Qualification Principle	6
3.1	Loss as an Extrinsic Quality of Motion	6
3.2	Qualified Efficiency	7
3.3	The Qualification Principle Applied Broadly	7
3.4	TOPSIS Distance: A Complementary Ranking	8
4	Metric I: Positional Leverage	9
5	Metric II: Instantaneous Functional Influence	9
6	Metric III: Structural Potential	10
6.1	Realized-to-Potential Ratio	11
7	Leverage-Weighted Source Attribution	12
8	Example-Level Attribution and the Learning Order Map	12
8.1	Motivation	12
8.2	Per-Example Efficiency Score	13
8.3	The Learning Order Map	13
8.4	The Four Example Types	14
8.5	Connection to Mechanistic Interpretability	14
9	Trajectory-Guided Hyperparameter Adaptation	15
9.1	The Observation	15
9.2	Finite-Difference Hyperparameter Gradient	15
9.3	Q as an Adaptive Stopping Signal	15
10	Summary Table	16

1 Base Accumulation: Gross Path and Net Displacement

Let $\theta_i(t)$ denote the value of parameter element i at training step t , and let $\Delta\theta_i(t) = \theta_i(t) - \theta_i(t-1)$ be the update applied at step t .

Gross path length.

$$G_i = \sum_t |\Delta\theta_i(t)| \quad (1)$$

The total distance parameter i has traveled through parameter space, counting every step in its absolute value. This is the *odometer reading*—it accumulates regardless of direction changes. A parameter that walks $+0.1$, then -0.1 , then $+0.1$, then -0.1 has $G_i = 0.4$.

Net displacement.

$$D_i = \sum_t \Delta\theta_i(t) = \theta_i(T) - \theta_i(0) \quad (2)$$

The signed sum of all updates: the straight-line vector from the initial position to the current position. The wandering parameter above has $D_i = 0$.

The G – D gap as foundational diagnostic. $G_i \geq |D_i|$, with equality only when the parameter walks a monotone straight line. Every higher-level metric in this document inherits this design: gross quantities measure work done (including wasted, oscillatory, or fought-over work); net quantities measure work committed to the trajectory; their ratios surface different kinds of inefficiency.

Network-level efficiency: two distinct ratios. Two questions are worth separating:

1. *Waste vs. purposefulness*: across all parameters, what fraction of total motion was directed rather than cancelled?
2. *Geodesic-ness*: how close was the network’s overall trajectory to a Euclidean straight line from $\theta(0)$ to $\theta(T)$?

These are answered by different norms:

$$\eta_1 = \frac{|D|_1}{G} = \frac{\sum_i |D_i|}{\sum_i G_i} \in [0, 1] \quad (3)$$

$$\eta_2 = \frac{|D|_2}{G} \in [0, 1] \quad (4)$$

where $G = \sum_i G_i$ is the total gross path. η_1 is the *waste metric*: it equals 1 when every parameter’s motion is perfectly directed, and approaches 0 when updates cancel within each parameter. η_2 is the *geodesic metric*: it additionally penalizes dispersion of directed motion *across* parameters, shrinking whenever D_i point in different directions even if each $|D_i|/G_i$ is large.

The relationship $\eta_2 \leq \eta_1 \leq 1$ holds always. The gap $\eta_1 - \eta_2$ measures dispersion of directed motion across parameters—structural information about how coordinated learning is, but not waste.

A key connection: η_1 is exactly the G_i -weighted mean of per-parameter efficiencies:

$$\eta_1 = \sum_i \frac{G_i}{G} \cdot \eta_i \quad (5)$$

η_2 does *not* admit this decomposition. Readers should not assume averaging η_i recovers either network-level ratio without checking which norm is in use.

Per-parameter efficiency.

$$\eta_i = \frac{|D_i|}{G_i} \in [0, 1] \quad (6)$$

$\eta_i \rightarrow 1$: the parameter walked a near-straight line. $\eta_i \rightarrow 0$: it churned without going anywhere. The distribution of $\{\eta_i\}$ across the network—and how it shifts by layer, parameter type, and gradient source—is a fine-grained diagnostic not captured by the network-level scalars.

1.1 Per-Source Attribution

When training uses multiple gradient sources (e.g., a trunk loss $\mathcal{L}_0$ and auxiliary head losses $\mathcal{L}_1, \dots, \mathcal{L}_H$), the total gradient at step t is

$$g_i(t) = \sum_{s=0}^{S-1} g_i^{(s)}(t),$$

where $S = H + 1$ is the total number of sources. A single Adam step is applied using the combined gradient $g_i(t)$, preserving unified moment estimates. The actual parameter update is then attributed proportionally:

$$\Delta\theta_i^{(s)}(t) = \frac{g_i^{(s)}(t)}{g_i(t)} \cdot \Delta\theta_i(t) \quad (7)$$

Approximation caveat. This proportional split is exact in the sense that pre-Adam source proportions are preserved post-Adam. However, Adam’s moment estimates $\hat{m}_i$ and $\hat{v}_i$ are accumulated on the *combined* gradient history. Per-step attribution is exact, but the integrated $D_i^{(s)}$ is path-dependent: a late-arriving source inherits the adaptive scaling geometry carved by sources that dominated earlier in training. The attribution should be read as “which source drove this step,” not “what would this source have done alone.”

Per-source gross path and net displacement:

$$G_i^{(s)} = \sum_t |\Delta\theta_i^{(s)}(t)| \quad (8)$$

$$D_i^{(s)} = \sum_t \Delta\theta_i^{(s)}(t) \quad (9)$$

By construction $\sum_s D_i^{(s)} = D_i$ exactly.

1.2 Per-Source Per-Parameter Efficiency

$$\eta_i^{(s)} = \frac{|D_i^{(s)}|}{G_i^{(s)}} \in [0, 1] \quad (10)$$

When $G_i^{(s)} = 0$, set $\eta_i^{(s)} = \text{NaN}$ and exclude from aggregations. This is the primary observable for the recruitment hypothesis: a parameter where the task gradient churns (low $\eta_i^{(0)}$) while the structured head gradient is directed (high $\eta_i^{(s)}$, $s > 0$) is one where the auxiliary signal is doing geodesic work the task signal cannot.

2 Steepness: The Extrinsic Dimension of Movement

2.1 The Core Observation

The efficiency metrics of Section 1 are *intrinsic*: they describe the geometry of the parameter trajectory without any reference to the loss landscape. A perfectly directed trajectory ($\eta_1 = 1$) earns a perfect intrinsic score whether it walks straight into a deep minimum or straight along a flat plateau.

The missing dimension is *extrinsic productivity*: how steeply did the loss descend per unit of parameter motion? A displacement vector has both direction and magnitude. The intrinsic metrics capture direction (was it coherent?). Steepness captures productive magnitude (did it go somewhere useful?).

Consider two training runs reaching the same final loss:

- Run A: $\eta_1 = 0.9$, loss improvement = 0.05. Highly directed, shallow descent.
- Run B: $\eta_1 = 0.4$, loss improvement = 0.80. Chaotic, but steep descent.

Run A is not obviously preferable. Efficient shallow descent may indicate the model is navigating a plateau precisely but not productively. Run B may be stumbling toward a deep minimum. The framework’s job is to make these trade-offs visible, not to resolve them.

2.2 Two Steepnesses

Steepness asks how much loss improvement a unit of movement purchased—but there are two natural denominators, corresponding to gross and net motion respectively, and they answer different questions.

Gross steepness σ_G : loss per unit of effort.

$$\sigma_G = \frac{\Delta\mathcal{L}}{G} = \frac{\mathcal{L}(0) - \mathcal{L}(T)}{G} \quad (11)$$

Loss improvement per unit of total gross parameter motion. This qualifies the odometer: of all the steps taken, including the wasted ones, how much did training descend? High σ_G means each unit of parameter work, even the oscillatory work, bought real loss improvement. This is the right denominator when asking about the productivity of *effort*.

Net steepness σ_D : loss per unit of committed displacement.

$$\sigma_D = \frac{\Delta\mathcal{L}}{|D|_1} \quad (12)$$

Loss improvement per unit of net displacement. This qualifies the straight-line movement: of the distance the model actually committed to traveling, how steeply did it descend? High σ_D means the committed direction was well-aimed. This is the right denominator when asking about the productivity of *direction*.

The relationship between them. Since $\eta_1 = |D|_1/G$, we have exactly:

$$\sigma_D = \frac{\sigma_G}{\eta_1} \quad (13)$$

A highly directed run ($\eta_1 \approx 1$) has $\sigma_D \approx \sigma_G$: most effort was committed, so effort-productivity and direction-productivity are nearly the same. A chaotic run ($\eta_1 \ll 1$) has $\sigma_D \gg \sigma_G$: the committed direction was actually steep, even though most effort was wasted on oscillation. The three quantities η_1 , σ_G , and σ_D are not independent—any two determine the third—but each is the natural answer to a distinct question.

Per-step versions. At training step t , using batch loss as a proxy for test loss (fast but conflates train/test):

$$\sigma_G(t) = \frac{\mathcal{L}(t-1) - \mathcal{L}(t)}{\sum_i |\Delta\theta_i(t)|} \quad \sigma_D(t) = \frac{\mathcal{L}(t-1) - \mathcal{L}(t)}{\sum_i |\Delta\theta_i^{\text{net}}(t)|} \quad (14)$$

Both can be negative on a bad step. Accumulated over training, $\sum_t \sigma_G(t) \cdot G(t) = \Delta\mathcal{L}_{\text{total}}$, recovering total loss improvement as a weighted sum of per-step gross steepnesses.

2.3 The Steepness–Directedness Decomposition

Together, σ_G (gross steepness) and η_1 (directedness) decompose the quality of a training run along two independent axes:

	High σ_G (steep)	Low σ_G (shallow)
High η_1 (directed)	Ideal: purposeful steep descent	Plateau navigation: efficient but unproductive
Low η_1 (chaotic)	Productive thrashing: noisy but learning	Wasted motion: neither directed nor improving

The “plateau navigation” region is particularly insidious: high η_1 can mask stagnation. The “productive thrashing” region (small-batch early training) is where most productive learning occurs in practice—the framework makes this visible rather than penalizing it as waste.

3 The Qualification Principle

3.1 Loss as an Extrinsic Quality of Motion

The intrinsic metrics (G , D , η_1 , η_2 , η_i) describe the geometry of the parameter trajectory. They are defined entirely in terms of parameter updates—no loss evaluations required. Their signal is real: a training run with high η_1 moved purposefully, regardless of whether that purposeful motion improved the model.

But purposeful motion and productive motion are different things. The loss landscape is the extrinsic judge: it does not care how internally consistent the parameter updates were; it only cares where the parameters ended up. The gap between directed motion and productive motion is exactly the gap between memorization and generalization—a network can move very purposefully toward a memorized solution that generalizes poorly.

The qualification principle: any intrinsic metric M admits a loss-qualified twin $M^\dagger = M \cdot \Delta^*$, where:

$$\Delta^* = \max\left(1 - \frac{\mathcal{L}_{\text{test}}(T)}{\mathcal{L}_{\text{test}}(0)}, 0\right) \in [0, 1] \quad (15)$$

is the fractional improvement in held-out test loss from initialization to end of training. $\Delta^* = 0$ means the model learned nothing (or degraded); $\Delta^* = 1$ means the model drove test loss to zero. The $\max(\cdot, 0)$ floor prevents negative values when test loss increases.

Why test loss. Using training loss for Δ^* rewards overfitting. A model can drive training loss to zero by memorizing; its Δ_{train}^* would be high while Δ_{test}^* reveals the problem. The framework is interested in learning quality.

Implementation note. Measure $\mathcal{L}_{\text{test}}(0)$ on the freshly initialized model before the optimizer takes any steps. This baseline is architecture-and-data-dependent but hyperparameter-independent—every configuration in a grid sweep shares the same baseline.

3.2 Qualified Efficiency

The primary qualified metric is *qualified efficiency*:

$$\boxed{Q = \eta_1 \cdot \Delta^*} \quad (16)$$

$Q \in [0, 1]$. A model that moved with perfect directedness but learned nothing scores $Q = 0$. A model that learned maximally but moved with maximum churn also scores $Q = 0$. The jointly optimal model—directed motion that produces steep loss improvement—scores near $Q = 1$.

The relationship to steepness is:

$$Q = \eta_1 \cdot \Delta^* \approx \eta_1 \cdot \frac{\sigma_G \cdot G}{\mathcal{L}(0)} \quad (17)$$

So Q is approximately the product of directedness and gross steepness, normalized by initial loss scale. The triple (η_1, σ_G, Q) is coherent: directedness, productivity-per-unit-effort, and their joint outcome.

3.3 The Qualification Principle Applied Broadly

The same Δ^* factor can be stacked onto any intrinsic metric to produce its qualified twin. A few canonical examples:

Qualified gross path.

$$G^\dagger = G \cdot \Delta^* \quad (18)$$

Total parameter work, discounted by how much of it produced real learning. High $G^\dagger$ requires both significant parameter movement and significant test-loss improvement.

Qualified net displacement.

$$D^\dagger = |D|_1 \cdot \Delta^* \quad (19)$$

Committed parameter movement, discounted by productivity. This is related to Q by $D^\dagger = Q \cdot G$: qualified efficiency times total gross path recovers qualified net displacement.

Qualified per-source efficiency.

$$Q^{(s)} = \eta_1^{(s)} \cdot \Delta^{*(s)} \quad (20)$$

where $\Delta^{*(s)}$ is the fractional loss improvement on the steps where source s dominated. This answers: when this gradient source was steering, how directed was the motion, and how productive were those steps?

Qualified leverage-weighted attribution.

$$\mathcal{B}^{(s)\dagger} = \mathcal{B}^{(s)} \cdot \Delta^* \quad (21)$$

Leverage-weighted gross motion by source, discounted by overall learning productivity. This qualifies ϕ and ψ (Section 7) in the same way.

The general rule. Any metric in this framework that describes how parameters moved can be multiplied by Δ^* to ask whether that movement was worth it. The intrinsic and qualified versions are both worth tracking; their ratio is Δ^* itself, which measures how much of the geometric behavior translated into actual learning.

3.4 TOPSIS Distance: A Complementary Ranking

For comparing configurations across a hyperparameter grid, a second scalar is useful: the *TOPSIS distance* to the ideal point ($\mathcal{L} = 0$, $\eta_1 = 1$). After normalizing both axes to $[0, 1]$ across the comparison set:

$$d_{\text{TOPSIS}} = \sqrt{\ell_{\text{norm}}^2 + (1 - \eta_{1,\text{norm}})^2} \quad (22)$$

Lower distance is better. Q and d_{TOPSIS} are complementary: Q is an absolute product (interpretable without a comparison set), while d_{TOPSIS} is a relative ranking (requires a set of configurations to normalize over). Both should be tracked.

Code note. For a grid sweep over batch size $\times$ learning rate: compute Q for each cell (absolute), then compute d_{TOPSIS} across the full grid for ranking. Plot both as heatmaps. The efficiency ridge—the locus of configurations with high Q —should appear as a diagonal band in the batch-size $\times$ LR space, consistent with the linear scaling rule relating optimal LR to batch size.

4 Metric I: Positional Leverage

$$\lambda_i^{\text{pos}} = \frac{|\{j : j \text{ is downstream of } i\}|}{P} \in [0, 1] \quad (23)$$

Pure graph topology: the fraction of parameters that depend on θ_i through the computation graph. Value-independent and loss-independent. A weight in an early layer feeds into many downstream computations and has high λ^{pos} ; a weight in the final layer has low λ^{pos} .

Vanishing gradient counterpoint. This metric and gradient magnitude point in opposite directions for early layers. Early parameters have maximum positional leverage but minimum gradient magnitude. Positional leverage measures *potential* influence regardless of whether the current values allow that influence to propagate.

$$\tilde{G}_i^{\text{I}} = \lambda_i^{\text{pos}} \cdot G_i \quad (24)$$

5 Metric II: Instantaneous Functional Influence

$$\lambda_i^{\text{fn}}(t) = \left\| \frac{\partial f}{\partial \theta_i}(t) \right\|_2^2 = \sum_{j=1}^p \left(\frac{\partial f_j}{\partial \theta_i}(t) \right)^2 \quad (25)$$

where f is the functional output of the network: $f(x) = \log \text{softmax}(z(x)) \in \mathbb{R}^C$ for classification, or $f(x) \in \mathbb{R}^D$ for regression, and $p = C$ or $p = D$ respectively. This is the diagonal of the Fisher Information Matrix evaluated at the current parameter values $\theta(t)$ —the total squared sensitivity of the model’s output to θ_i right now, summed symmetrically over all output dimensions without reference to any label, loss, or argmax.

Value-dependent and live: unlike λ_i^{str} , which averages out initialization-dependence to expose architectural structure, $\lambda_i^{\text{fn}}(t)$ is a snapshot. It reflects not just where θ_i sits in the graph but what the current weight configuration allows it to do. The same parameter position can have very different λ_i^{fn} at different points in training.

Relationship to the training gradient.

$$\frac{\partial \mathcal{L}}{\partial \theta_i} = \frac{\partial \mathcal{L}}{\partial f} \cdot \frac{\partial f}{\partial \theta_i}$$

Functional influence is the “structural half” of the loss gradient—the part depending on the parameter’s role in the function, separated from the part depending on the loss geometry. Vanishing gradients arise from either a flat loss surface ($\partial \mathcal{L} / \partial f$ small) or a small Jacobian ($\partial f / \partial \theta_i$ small). Metric II isolates the second factor, making it diagnosable independently of the loss.

Riemannian interpretation. The Fisher Information Matrix $\mathcal{F} = \mathbb{E}[J^\top J]$, where $J = \partial f / \partial \theta$, is the natural Riemannian metric tensor on parameter space. Moving in direction v costs $\sqrt{v^\top \mathcal{F} v}$ under this metric. $\lambda_i^{\text{fn}}(t)$ is the i -th diagonal entry of $\mathcal{F}$ at the current parameter values—a diagonal approximation to the natural metric, computed without ever materializing the full matrix.

Computation.

$$\lambda_i^{\text{fn}}(t) \approx \frac{1}{|X_{\text{probe}}|} \sum_{x \in X_{\text{probe}}} \sum_{j=1}^p \left(\frac{\partial f_j(x)}{\partial \theta_i}(t) \right)^2 \quad (26)$$

Evaluated at checkpoints, not inside the training loop. The same probe set X_{probe} used for λ_i^{str} should be reused here for consistency: both metrics are then expectations over the same input distribution, so $\rho_i(t) = \lambda_i^{\text{fn}}(t)/\lambda_i^{\text{str}}$ is a clean ratio without confounding from different x draws. This requires p backward passes per checkpoint per probe example—run at coarse intervals, not every step.

Data dependence. Because $\lambda_i^{\text{fn}}(t)$ is not averaged over initializations, it inherits variance from both the current weight values and the probe inputs. A single representative example is riskier here than for λ_i^{str} : the current network may have learned input-specific structure that makes a single x a poor summary. The probe set average is more important here than in the structural case.

$$\tilde{G}_i^{\text{II}}(t) = \lambda_i^{\text{fn}}(t) \cdot G_i(t) \quad (27)$$

6 Metric III: Structural Potential

$$\lambda_i^{\text{str}} = \mathbb{E}_{\substack{\theta \sim P_{\text{init}} \\ x \sim P_{\text{data}}}} \left[\left\| \frac{\partial f}{\partial \theta_i} \right\|_2^2 \right] \quad (28)$$

where f is the functional output of the network: $f(x) = \log \text{softmax}(z(x)) \in \mathbb{R}^C$ for classification, or $f(x) \in \mathbb{R}^D$ for regression. The squared norm then expands as:

$$\left\| \frac{\partial f}{\partial \theta_i} \right\|_2^2 = \sum_{j=1}^p \left(\frac{\partial f_j}{\partial \theta_i} \right)^2 \quad (29)$$

where $p = C$ (output classes) for classification and $p = D$ (output dimensions) for regression. This is the diagonal of the Fisher Information Matrix evaluated at θ —or equivalently, the total squared sensitivity of the model’s output distribution to θ_i , summed symmetrically over all output dimensions. No labels, no loss, no argmax: f_j is simply the j -th component of the network’s functional output.

The expected Jacobian norm of parameter θ_i under the initialization distribution and the data distribution. You cannot do calculus without values—but you can average over values. As you average over many random initializations and input examples, idiosyncratic value- and example-dependent fluctuations cancel and the structural signal persists: the mean propagation gain from position i to the output, determined by layer type, activation function, depth, and connectivity.

Computation.

$$\lambda_i^{\text{str}} \approx \frac{1}{K} \sum_{k=1}^K \frac{1}{|X_{\text{probe}}|} \sum_{x \in X_{\text{probe}}} \left\| \frac{\partial f^{(k)}(x)}{\partial \theta_i} \right\|_2^2 \quad (30)$$

Computed *once* before training begins; reused as a fixed scaling factor throughout. The expectation is approximated jointly over K random initializations and a fixed probe set X_{probe} drawn once from the data distribution. A probe set of 64–256 examples is the practical sweet spot: large enough to suppress input-dependent variance, small enough that K full Jacobian evaluations remain tractable. Using the full dataset is correct but rarely necessary; using a single mean feature vector is fast but can miss activation-pattern variance across examples.

Heavy precomputation (large K) is warranted. Report standard error $\sigma_i/\sqrt{K}$ alongside the mean; flag parameters where median and mean diverge substantially (heavy-tailed distributions near saturating nonlinearities). For flagged parameters, use a trimmed mean as the denominator of $\rho_i(t)$.

Network-level leverage budget. Define the total structural leverage of the network:

$$\Lambda = \sum_i \lambda_i^{\text{str}} \quad (31)$$

and the per-parameter leverage share:

$$\pi_i = \frac{\lambda_i^{\text{str}}}{\Lambda} \in [0, 1], \quad \sum_i \pi_i = 1 \quad (32)$$

π_i is the fraction of the network’s total functional leverage residing in parameter θ_i . It defines a probability distribution over parameters—the architecture’s intrinsic prior on which parameters matter most. Layer-level aggregates $\pi^{(\ell)} = \sum_{i \in \ell} \pi_i$ give a leverage budget per layer: what share of the network’s total functional surface area does each layer control?

Connection to mean field theory. Poole et al. (2016) derived the per-layer analog analytically, showing exponential decay with depth in the “ordered phase” and growth in the “chaotic phase,” with a critical “edge of chaos” boundary. Metric III is the empirical per-parameter version of the same quantity, applicable to arbitrary architectures without analytical tractability assumptions.

$$\tilde{G}_i^{\text{III}} = \lambda_i^{\text{str}} \cdot G_i \quad (33)$$

6.1 Realized-to-Potential Ratio

$$\rho_i(t) = \frac{\lambda_i^{\text{fn}}(t)}{\lambda_i^{\text{str}}} \quad (34)$$

$\rho_i \approx 1$: the parameter is being used roughly as the architecture intends. $\rho_i \ll 1$: suppression—high structural potential not currently realized (vanishing gradients, saturated activations, dead units). $\rho_i \gg 1$: amplification—the parameter is in a “hot” configuration, exerting more functional influence than its architectural average predicts.

Because λ_i^{str} is fixed and $\lambda_i^{\text{fn}}(t)$ is live, $\rho_i(t)$ cleanly separates what the architecture affords from what training has realized. A parameter with high π_i and low $\rho_i(t)$ is structurally important but currently suppressed—a candidate for targeted intervention. A parameter with low π_i and high $\rho_i(t)$ is being over-recruited relative to its architectural role.

Watching $\rho_i(t)$ evolve over training, decomposed by gradient source, reveals which training objectives recruit structurally important parameters and which spend budget on structurally peripheral ones.

7 Leverage-Weighted Source Attribution

Given leverage weights λ_i (from any of Metrics I–III):

$$\mathcal{B}^{(s)} = \sum_i \lambda_i \cdot G_i^{(s)} \quad (35)$$

The total leveraged motion source s has produced. A source that pushed hard at low-leverage parameters has small $\mathcal{B}^{(s)}$; one that pushed at high-leverage parameters has large $\mathcal{B}^{(s)}$.

$$\tilde{D}^{(s)} = \left| \lambda \odot D^{(s)} \right|_2 \quad (36)$$

$$\tilde{\eta}^{(s)} = \frac{\tilde{D}^{(s)}}{\mathcal{B}^{(s)}} \quad (37)$$

Two normalizations of structured-source share:

$$\phi = \frac{\mathcal{B}^{(\text{struct})}}{\sum_s \mathcal{B}^{(s)}} \quad (38)$$

$$\psi = \frac{\mathcal{B}^{(\text{struct})}}{\sum_i \lambda_i \cdot G_i} \quad (39)$$

ϕ normalizes against all demanded leveraged motion (including cancelled work between sources). ψ normalizes against actually committed leveraged motion. $\psi > 1$ is possible and informative: the structured heads pushed for more leveraged motion than the trajectory committed to, meaning their pushes were partially cancelled. The gap $\psi - \phi$ surfaces the degree of inter-source conflict. Track both; they answer different questions.

If ϕ rises as Q rises, the structured heads are the geodesic force.

8 Example-Level Attribution and the Learning Order Map

8.1 Motivation

Everything in Sections 1–7 attributes motion and efficiency to *gradient sources* (which loss function, which head). A complementary and deeper attribution is possible: to *training examples*. Just as backpropagation assigns credit for loss to specific parameters, we can assign credit for learning progress to specific data points.

This is inspired by Prioritized Experience Replay (PER, Schaul et al. 2015), which maintains a per-example priority score and samples high-priority examples more frequently. PER defines priority by current loss magnitude—the highest-loss examples are replayed most. The framework here defines an analogous per-example score grounded in trajectory geometry: *on which examples did the model make steep, well-aimed moves?*

8.2 Per-Example Efficiency Score

After each batch update at step t , the trajectory metrics for that step are available:

- $\eta_1(t)$: directedness of the parameter update
- $\sigma_G(t)$: gross steepness (loss drop per unit gross motion)
- $\sum_i \lambda_i \cdot |\Delta\theta_i(t)|$: leverage-weighted motion magnitude

For each example x_n present in the batch at step t , accumulate a scalar score:

$$\xi_n(t) = \frac{1}{|\mathcal{B}_t|} \cdot Q(t) \cdot \sum_i \lambda_i \cdot |\Delta\theta_i(t)| \quad (40)$$

where $\mathcal{B}_t$ is the batch at step t and the $1/|\mathcal{B}_t|$ factor distributes the step’s credit equally among all examples in the batch. The cumulative per-example score is:

$$\Xi_n = \sum_{t:n \in \mathcal{B}_t} \xi_n(t) \quad (41)$$

This is the total leverage-weighted, loss-qualified, directedness-weighted learning credit attributed to example x_n over training.

Interpretation.

- **High Ξ_n :** steps containing x_n were steep, directed, and leverage-productively weighted. x_n is a *high-yield example*—the model was primed to learn from it when it appeared.
- **Low Ξ_n :** steps containing x_n were chaotic, shallow, or both. x_n is either inherently hard or arrives at the wrong moment in training.
- **Decreasing $\xi_n(t)$ over time:** x_n was productive early but the model has saturated its information content—it taught something specific and is now redundant.
- **Increasing $\xi_n(t)$ over time:** x_n was unlearnable early but becomes highly productive late—its information required prerequisite structure the model had not yet built.

8.3 The Learning Order Map

Track $\xi_n(t)$ for all examples and epochs, producing a matrix $\Xi \in \mathbb{R}^{N \times T}$ where N is the dataset size and T is the number of training epochs. This matrix encodes the *temporal structure of learning*—which examples yielded productive steps, and when.

Analysis operations on Ξ .

1. **Row clustering:** group examples by their temporal efficiency profile. Examples with similar $\xi_n(\cdot)$ curves belong to the same *learning family*—learned together because they share a common structure the model discovered at a particular phase.

2. **Column (epoch) analysis:** the epoch-level mean $\bar{\xi}(t) = \frac{1}{N} \sum_n \xi_n(t)$ tracks how productive training was as a function of time. A spike in $\bar{\xi}(t)$ indicates a phase where many examples suddenly became productive—this is the trajectory-metric signature of grokking.
3. **Correlation with input structure:** in the modular arithmetic setting, examples are fully characterized by (a, b) . Plotting Ξ_n as a function of a and b reveals which arithmetic relationships were learned first and which were learned last.
4. **Prerequisite detection:** if example x_n has a delayed spike in $\xi_n(t)$, examine which examples $\{x_m\}$ had high $\xi_m(t')$ for $t' < t$. These are the candidate *prerequisites*—examples that built the structure x_n required.

8.4 The Four Example Types

The steepness–directedness decomposition applied at the example level produces a taxonomy:

	High $\sigma_{G,n}$ (steep steps)	Low $\sigma_{G,n}$ (shallow steps)
High $\eta_{1,n}$ (directed)	<i>Gateway:</i> model learns cleanly and productively	<i>Plateau:</i> purposeful motion, little loss improvement
Low $\eta_{1,n}$ (chaotic)	<i>Struggle:</i> thrashing but still descending	<i>Wasted:</i> neither directed nor productive

Gateway examples are the highest-yield training data: they appear at the right time, align with what the model is currently capable of learning, and produce steep directed improvement. A curriculum that preferentially samples gateway examples would accelerate learning.

Struggle examples correspond to the PER high-priority class—the model loses a lot on them and updates are noisy. But the trajectory framework distinguishes between struggle examples that are nevertheless descending the loss (productive thrashing) and those that are merely churning. PER cannot make this distinction; the trajectory framework can.

8.5 Connection to Mechanistic Interpretability

The learning order map is mechanistic interpretability from the data side rather than the activation side. Instead of asking “what circuit fired for this example,” it asks “when did this family of examples become efficiently learnable.” These questions are complementary and should converge: a circuit that crystallizes at epoch t_0 should correspond to a cohort of examples whose $\xi_n(t)$ rises sharply at t_0 .

For the modular arithmetic grokking experiment this is directly testable. Nanda et al. (2023) identified the Fourier features and the epoch at which they crystallize. The learning order map should show a cohort of examples whose $\xi_n(t)$ spikes at exactly that epoch. If it does, activation-space interpretability and trajectory-space attribution are confirmed to be seeing the same event from different angles.

9 Trajectory-Guided Hyperparameter Adaptation

9.1 The Observation

Empirically, Q depends on both the hyperparameters and the current state of training. In the early phase, small-batch small-LR configurations produce highest Q : the model is in a chaotic region and low-variance gradient estimates with small step sizes navigate it more productively. In the late phase, larger batches and higher learning rates can sustain high Q because the model has found stable descent directions and can be pushed harder along them.

This suggests that optimal hyperparameters are not static but *trajectory-dependent*: the best batch size and learning rate at epoch t depend on the current $Q(t)$ and its recent history. Standard schedules (cosine decay, cyclical LR) are heuristic approximations to this ideal.

9.2 Finite-Difference Hyperparameter Gradient

Let $Q(h, t)$ denote the qualified efficiency at training step t under hyperparameter configuration $h = (\text{bs}, \text{lr})$. The ideal update is:

$$h_{t+1} = h_t + \alpha_h \cdot \nabla_h Q(h_t, t) \quad (42)$$

The full gradient $\nabla_h Q$ is intractable analytically. A tractable approximation uses finite differences over short horizons:

1. At step t , run N_{probe} steps under h_t ; measure $\bar{Q}_0 = \frac{1}{N_{\text{probe}}} \sum Q(t+k)$.
2. Perturb: run N_{probe} steps under $h_t + \epsilon \hat{e}_{\text{lr}}$; measure $\bar{Q}_{\text{lr}}$.
3. Repeat for $h_t + \epsilon \hat{e}_{\text{bs}}$; measure $\bar{Q}_{\text{bs}}$.
4. Estimate $\nabla_h Q \approx [(\bar{Q}_{\text{lr}} - \bar{Q}_0)/\epsilon, (\bar{Q}_{\text{bs}} - \bar{Q}_0)/\epsilon]$.
5. Update h by a small step in the direction of increasing Q .

This is an online hyperparameter optimization algorithm guided by trajectory metrics rather than validation loss. The key difference: we optimize for *the quality of the learning process*, not just its outcome.

Practical constraint. Probe runs can be run on a small pilot batch rather than the full dataset, trading estimate variance for compute efficiency.

9.3 Q as an Adaptive Stopping Signal

Rather than stopping when validation loss plateaus, stop when Q has been consistently low for M consecutive epochs:

$$\text{stop if } \frac{1}{M} \sum_{k=0}^{M-1} Q(T-k) < Q_{\min} \quad (43)$$

This stops training when neither directedness nor productivity is being achieved. It is more principled than loss plateau detection because it requires *both* conditions to fail simultaneously.

10 Summary Table

Metric	Val.-dep.	Loss-dep.	What it measures
<i>Layers 1–3: Intrinsic (no loss evaluations required)</i>			
G_i, D_i	Yes	No	Odometer and displacement per parameter
$\eta_i = D_i /G_i$	Yes	No	Per-parameter directedness
$\eta_1 = D _1/G$	Yes	No	Network-wide waste (G_i -wtd. mean of η_i)
$\eta_2 = D _2/G$	Yes	No	Network-wide geodesic-ness
$\eta_i^{(s)}$	Yes	No	Per-param per-source directedness
$G_i^{(s)}, D_i^{(s)}$	Yes	No	Gross path and displacement per source
$\mathcal{B}^{(s)}, \phi, \psi$	Yes	No	Leveraged motion share per source
λ_i^{pos}	No	No	Structural potential by graph topology
$\lambda_i^{\text{fn}}(t)$	Yes	No	Realized behavioral influence now
λ_i^{str}	No (avg.)	No	Architectural influence in expectation
$\rho_i(t)$	Yes	No	Realized vs. architectural potential
<i>Layer 4: Extrinsic qualification (loss-dependent)</i>			
Δ^*	Yes	Yes	Fractional test-loss improvement
$\sigma_G = \Delta\mathcal{L}/G$	Yes	Yes	Gross steepness: loss per unit effort
$\sigma_D = \Delta\mathcal{L}/ D _1$	Yes	Yes	Net steepness: loss per unit direction
$Q = \eta_1 \cdot \Delta^*$	Yes	Yes	Qualified efficiency
$Q^{(s)} = \eta_1^{(s)} \cdot \Delta^{*(s)}$	Yes	Yes	Per-source qualified efficiency
$M^\dagger = M \cdot \Delta^*$	Yes	Yes	Any intrinsic metric, loss-qualified
d_{TOPSIS}	Yes	Yes	Distance to ideal ($\mathcal{L} = 0, \eta_1 = 1$)
<i>Layer 5: Example attribution</i>			
ξ_n	Yes	Yes	Per-example step-level learning credit
$\Xi \in \mathbb{R}^{N \times T}$	Yes	Yes	Learning order map

The two tensions, restated. All diagnostic value in this framework lives in one of two gaps:

1. G vs. $|D|$: the geometric gap. How much of the motion was wasted in oscillation? Captured by $\eta_1, \eta_2, \eta_i, \eta_i^{(s)}$.
2. η_1 vs. Q (equivalently, 1 vs. Δ^*): the productive gap. How much of the directed motion translated into actual learning? Captured by σ_G, σ_D, Q , and all qualified twins $M^\dagger$.

The geometric gap is intrinsic—measurable without any loss evaluations. The productive gap is extrinsic—requires held-out loss. Both are necessary; neither alone is sufficient.